CENTRO UNIVERSITARIO · LABORATORIO DE CÓMPUTO

PRÉSTAMOS LAB

Documentación de la aplicación web

Dato	Información
Estudiante	Carlos Fernando Ligorría Flores
Carné	0902-22-7467
Variante	Último dígito 7 · Monitor, Teclado y Mouse
Fecha	21 de octubre de 2026

React 19 · TypeScript · Vite · json-server

1. Resumen del proyecto

Préstamos Lab permite consultar el inventario del laboratorio, registrar quién se lleva cada equipo y controlar su fecha de devolución. La interfaz consume una API REST local y persiste los cambios en db.json mediante json-server.

Variante personal

Regla	Implementación
Carné terminado en 7	Variante correspondiente a los dígitos 6 o 7.
Categorías	Monitor, Teclado y Mouse; diez equipos de prueba.
Plazo máximo	Cinco días; el formulario limita y valida la fecha.
Regla adicional	El encabezado calcula y muestra los préstamos activos vencidos.
Título	Préstamos Lab — 0902-22-7467 en el encabezado y la pestaña.

Funcionalidades

- Tarjetas con disponibilidad; equipo ocupado no se puede prestar.
- Búsqueda por nombre y filtro de categoría combinados.
- Formulario controlado con errores de validación junto al campo.
- Tabla de préstamos activos, vencidos resaltados y acción Devolver.
- Carga con useEffect, indicador de progreso, error comprensible y reintento.
- Actualizaciones REST, estado inmutable y protección ante envíos dobles.

2. Componentes y organización

Presentación, vistas, acceso a datos y tipos están separados para mantener responsabilidades claras y props tipadas.

Ruta	Responsabilidad
src/components/Encabezado.tsx	Título y contadores derivados.
src/components/TarjetaEquipo.tsx	Tarjeta tipada y acción de préstamo.
src/components/ListaEquipos.tsx	Lista con map y key estable: equipo.id.
src/components/FormularioPrestamo.tsx	Formulario y validaciones.
src/components/TablaPrestamos.tsx	Tabla, vencimientos y devoluciones.
src/pages/Inventario.tsx	Vista y filtros combinados.
src/pages/PrestamosActivos.tsx	Vista de préstamos y resumen vencido.
src/services/api.ts	Funciones fetch tipadas y errores HTTP/red.
src/hooks/useEquipos.ts	Carga de equipos, loading, error y reintento.
src/types/index.ts	Interfaces Equipo y Prestamo.

Modelo de datos

Entidad	Campos
Equipo	id: number; nombre: string; categoria: string; disponible: boolean.
Prestamo	id: number; equipoId: number; solicitante: string; carne: string; fechaPrestamo: string (ISO 8601); fechaLimite: string (ISO 8601); devuelto: boolean.

3. Formularios, estado y API REST

Campo	Validación
Solicitante	Obligatorio, mínimo 5 caracteres.
Carné	Expresión regular ^\d{4}-\d{2}-\d{5}$; ejemplo 0000-00-00000.
Fecha límite	Desde hoy hasta un máximo de 5 días.

Los inputs se controlan con value y onChange. Los arreglos se actualizan inmutablemente con map/filter/spread, nunca mediante mutación directa. El envío se bloquea mientras la petición está en curso.

Método	Endpoint	Uso
GET	/equipos	Cargar inventario.
GET	/prestamos?devuelto=false	Cargar préstamos activos.
POST	/prestamos	Registrar un préstamo.
PATCH	/equipos/:id	Cambiar disponibilidad.
PATCH	/prestamos/:id	Marcar como devuelto.

La URL base proviene de VITE_API_URL (por defecto http://localhost:3001). Si falla una de las dos escrituras coordinadas, se intenta revertir la primera para evitar un estado inconsistente.

4. Instalación y ejecución

1. Instalar Node.js 20.19+ o 22.12+ y npm.
2. Ejecutar npm install en la raíz del proyecto.
3. En una terminal, ejecutar npm run api (API en http://localhost:3001).
4. En otra terminal, ejecutar npm run dev y abrir la URL indicada por Vite.
5. Comprobar la API en /equipos o /prestamos?devuelto=false, puerto 3001.

Comando	Descripción
npm run api	Inicia json-server; guarda cambios en db.json.
npm run dev	Inicia Vite.
npm run build	Valida TypeScript y genera dist/.
npm run lint	Ejecuta Oxlint.

Repositorio y commits

Repositorio público de la entrega: https://github.com/cligorriaf-sketch/gestor-prestamos-lab

El repositorio es público y la rama main contiene siete commits de avance, superando el mínimo de seis solicitado. El historial documenta el scaffold, las vistas y componentes tipados, la integración REST, los flujos de préstamo, la guía de ejecución, las evidencias y el enlace final.

#	Commit local	Etapa
1	chore: scaffold React TypeScript app with Vite	Estructura base y dependencias.
2	feat: add typed inventory and active loan views	Componentes y vistas.
3	feat: connect app to REST API with reusable data hook	Servicios y hook.
4	feat: implement loan workflows and responsive styling	Flujos y estilos.
5	docs: add API fixture and project setup guide	Datos y README.
6	docs: create assignment report and evidence screenshots	Informe y capturas.
7	docs: add public repository link to	Enlace al repositorio y documento

	final report	final publicados.

5. Regla adicional y reflexiones

La regla de la variante 6/7 se implementa como estado derivado. Una función compara la fecha ISO de cada préstamo activo con la fecha local de hoy y cuenta los que vencieron. El encabezado muestra ese total; la tabla resalta la fila vencida. Al devolver un préstamo, deja de formar parte del arreglo activo y del contador. No se almacena un contador duplicado en useState.

¿Por qué no conviene usar el índice como key si la lista puede cambiar?

Al insertar, eliminar o reordenar, el índice cambia de significado y React puede asociar el DOM o estado local a otro equipo. Una key estable como id permite reconciliar correctamente cada elemento.

¿Qué problema aparece si se modifica directamente el arreglo del estado?

La referencia no cambia y React puede no detectar que debe renderizar. La mutación también hace más difícil predecir el estado y rastrear errores; crear un arreglo nuevo con map, filter o spread evita esos problemas.

¿Qué ocurre si se omite el arreglo de dependencias en un useEffect que hace fetch y actualiza estado?

El efecto se ejecuta después de cada render. El fetch actualiza el estado, provoca otro render y puede iniciar peticiones repetidas o un ciclo continuo.

6. Evidencia de la aplicación

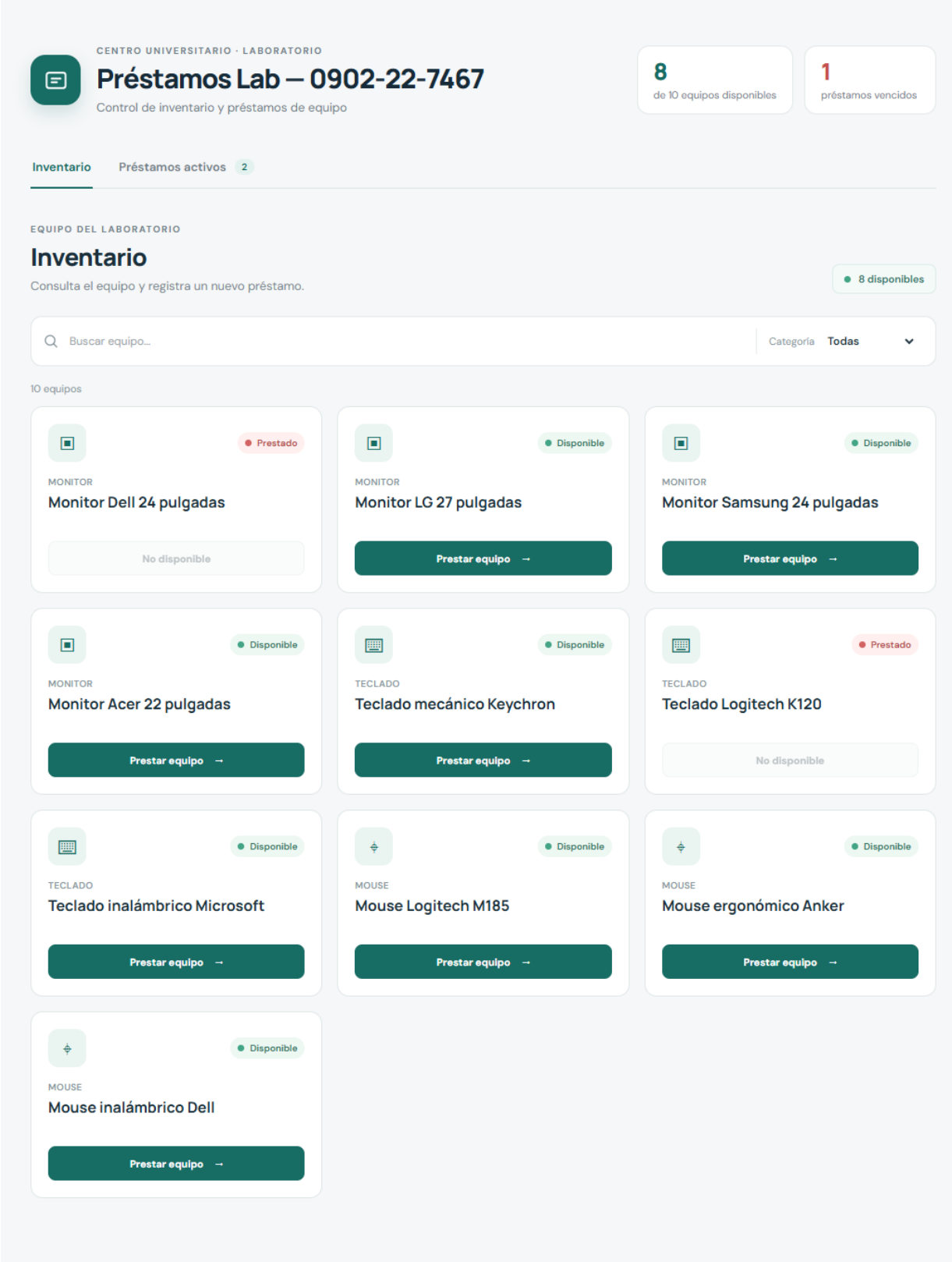


Figura 1. Inventario, filtros combinados, categorías y disponibilidad.

6. Evidencia de la aplicación

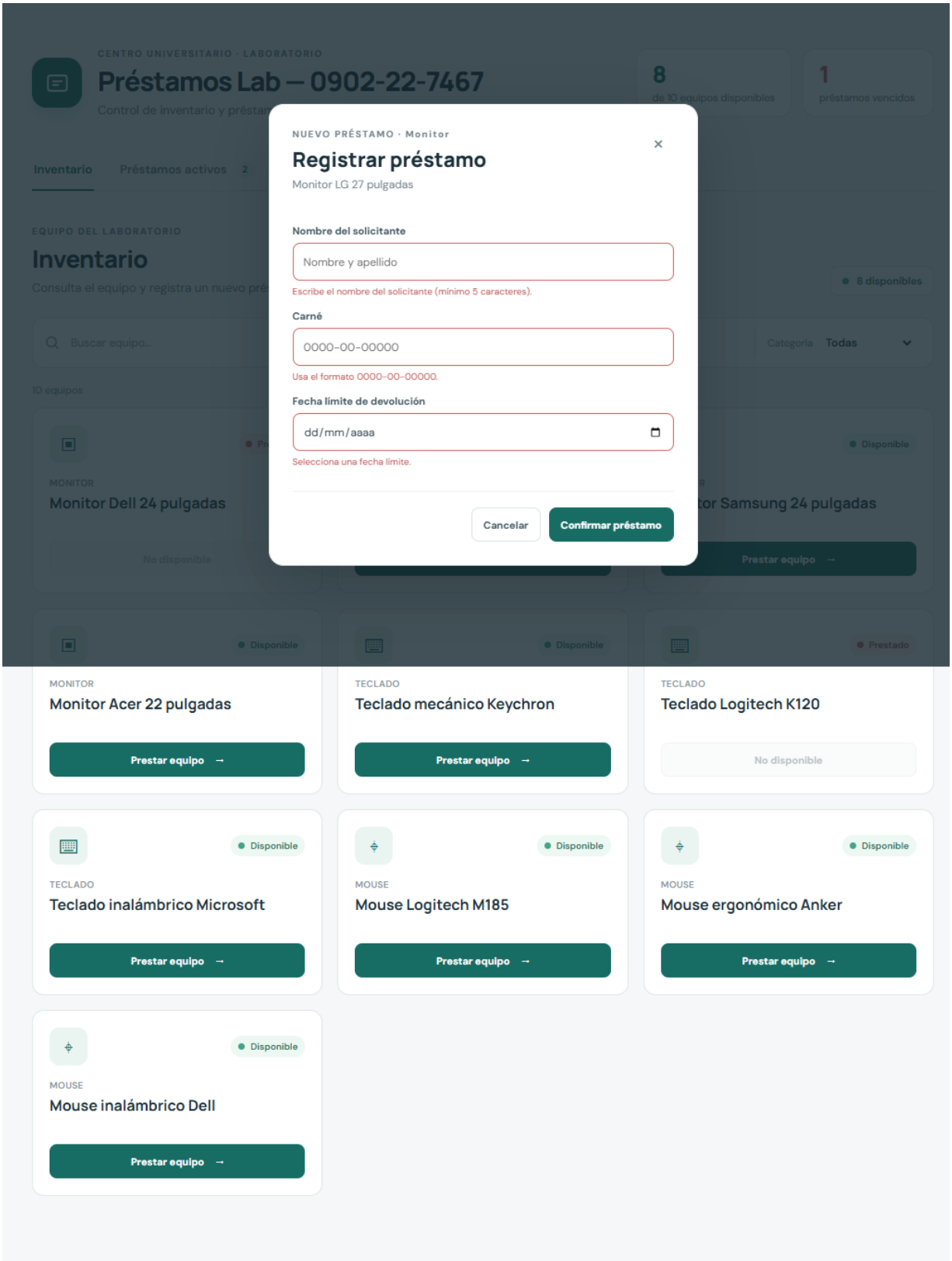


Figura 2. Errores de validación visibles para solicitante, carné y fecha.

6. Evidencia de la aplicación

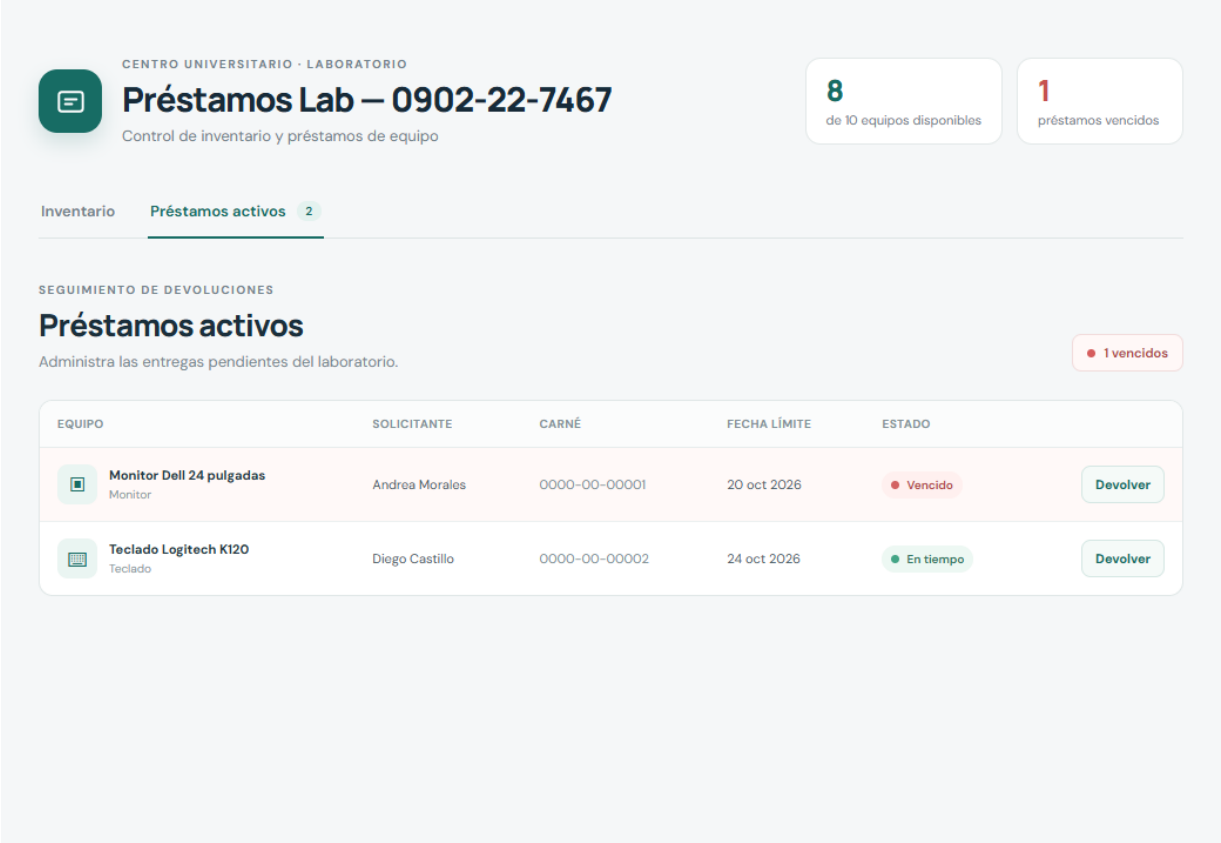


Figura 3. Préstamos activos con fila vencida resaltada y contador.

6. Evidencia de la aplicación

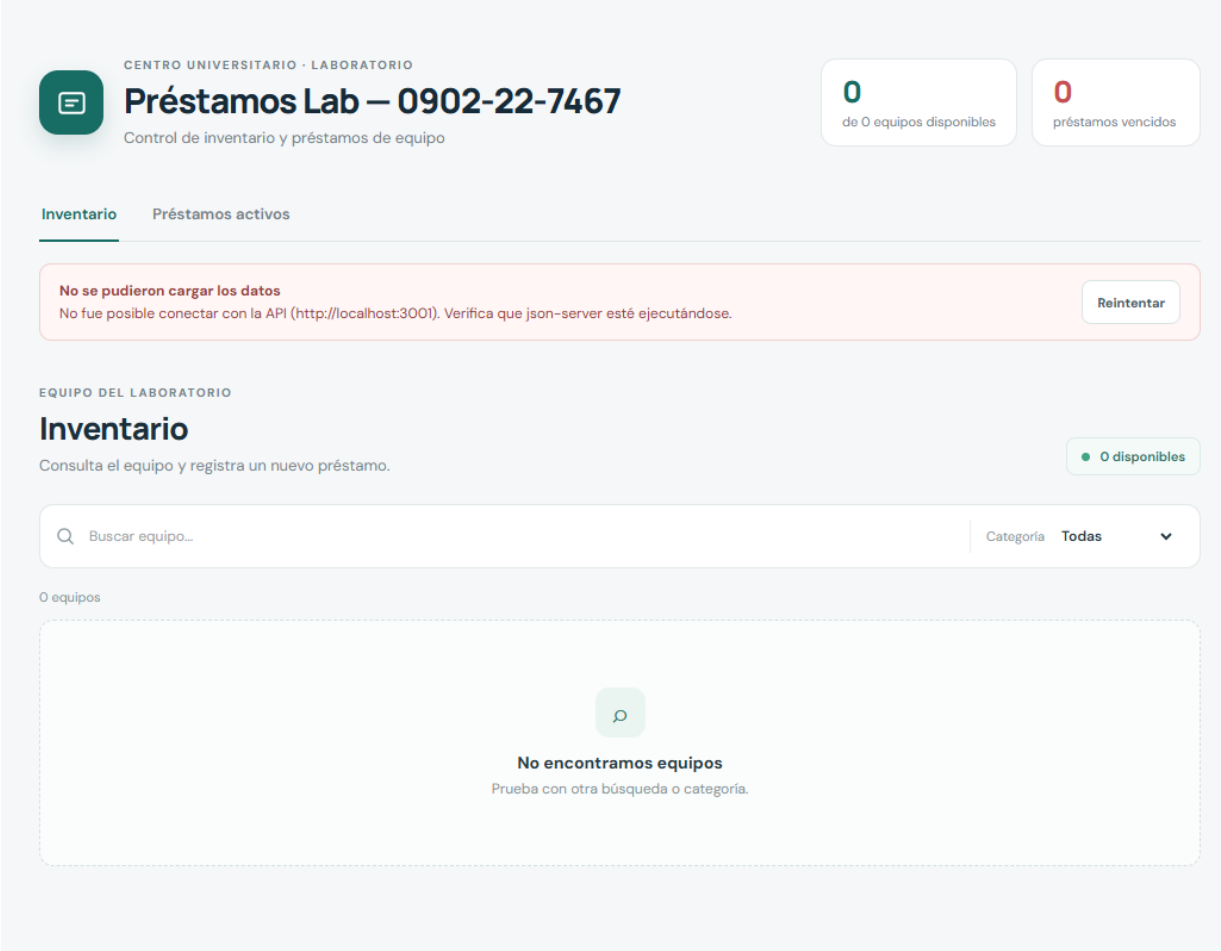


Figura 4. Error de conexión con API y botón Reintentar.